Database Management Systems
Comprehensive Notes

Ankur Gupta
www.ankurgupta.net

Contents

Chapter 1

Introduction to Database Systems

1.1 Database Management Systems

A database management system (DBMS) is a collection of programs that enables users to create and maintain a database.

1.2 Characteristics of the Database Approach

The main characteristics of the database approach are the following:

(1) Self-describing nature of a database system

(2) Insulation between programs and data, and data abstraction

(3) Support of multiple views of the data

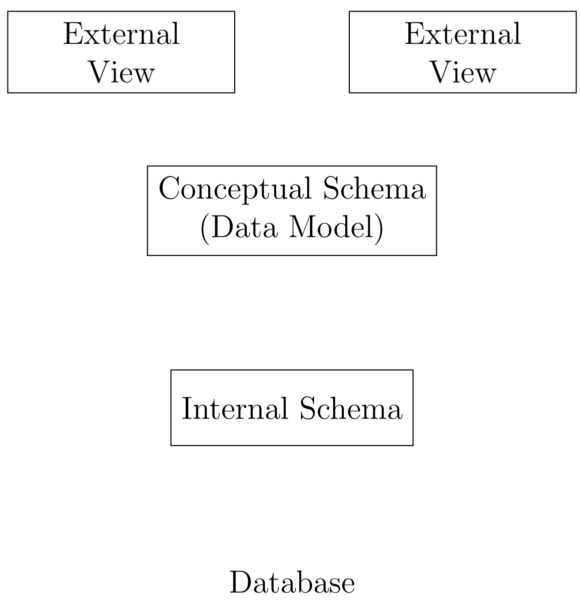
(4) Storing of data and multiuser transaction processing

Chapter 2

Database Architecture

2.1 The Three-Schema Architecture

The goal of the three-schema architecture is to separate the user applications and the physical database.



2.1.1 Schema Levels

(1) The internal level has an internal schema which describes the physical storage structure of the database

(2) The conceptual level has a conceptual schema which describes the structure of the whole database for a community of users

(3) The external or view level includes a number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group

2.2 Data Independence

Data independence can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

2.2.1 Types of Data Independence

(1) **Logical data independence:** Capacity to change the conceptual schema without having to change external schemas or user programs

(2) **Physical data independence:** Capacity to change the internal schema without having to change the internal conceptual schema

Chapter 3

Database Design

3.1 Database Design Techniques

3.1.1 Top-down Approach

We start by defining the data set and then we go on defining data elements in the set.

Example: Entity-Relationship Modeling

3.1.2 Bottom-up Approach

We start by defining required attributes first and then group those attributes to form the entities.

Example: Normalization

3.2 Database Design Approach

3.2.1 Entity-Relationship Modeling

The process of creating subclasses out of a given entity type is called specialization.

The reverse process of taking two or more entity types and clubbing them under a common super class is called generalization.

Chapter 4

Conceptual Design

4.1 Conceptual Schema

4.1.1 Entity

An entity represents an object of the real world that has an independent existence.

An entity has attributes or properties that describes it.

4.1.2 Attribute Types

Simple versus Composite

A composite attribute is made of one or more simple or composite attributes.

Example: Name is made of first name and last name.

Single-valued versus Multivalued

A single-valued attribute is described by one value (e.g., age of a person). A multivalued attribute is described by many values (e.g., the color of a multicolored bird).

Stored versus Derived

An attribute whose value can be inferred from the value of other attributes is called a derived attribute (e.g., age can be derived from date of birth and current date). If this is not the case, then it is a stored attribute.

Null Values

Attributes that do not have any applicable values are said to have null values. Different from 0, unknown, and missing values.

Complex Attributes

Combination of composite and multivalued attributes. For example, a person can have more than one address and each address can have different parts.

4.1.3 Entity Types and Entity Sets

An **entity type** defines a collection of different entities having the same type.

Example: "Department" defined earlier is an entity type defining several departments having the same attribute structure.

Any specific collection of entities of a particular type is called an entity set.

An entity type describes the schema or intension for an entity set.

4.1.4 Key Attributes

An attribute or a set of attributes that uniquely identify entities in an entity set are called **key attributes**.

When a collection of attributes define the key, the set is called a **composite key**.

Composite keys should be minimal. No subset of a composite key should be a key itself.

Key attributes are shown underlined in the Entity-Relationship (ER) Diagram.

Key attributes should retain the unique definition property for all extensions (EER model) of the entity type.

There may be more than one key in a relation. All such keys are known as **candidate keys**. One of the candidate keys is chosen as the **primary key**; the others are known as **alternate keys**.

4.1.5 Domains of Attributes

The domain of an attribute is the set of all valid values that the attribute can have.

Example: The domain of the attribute "age" of an employee is the set of all integers between 18 and 65.

If the composite attribute can have values with domains $D_1, D_2, \ldots, D_n$ then the domain of the attribute is:

$$D_1 \times D_2 \times \ldots \times D_n$$

4.2 Relationships

Relationship types relate two (or more) entity types and defines a set of associations (a relationship set) among entities of this type.

Formally:

$$R \subseteq E_1 \times E_2 \times \ldots \times E_n$$

A relationship instance is any $r_i \in R_i$ where $r_i = (e_1, e_2, \ldots, e_n)$ such that:

$$e_1 \in E_1,\ e_2 \in E_2,\ \ldots,\ e_n \in E_n$$

4.2.1 Degree of a Relationship Type

The number of participating entity types in this relationship type.

Example: Degree of the relationship "department managed by Employee" is 2.

Unary, binary, and ternary relationships are special forms for relationships of degree 1, 2, and 3, respectively.

4.3 Relationships versus Attributes

Relationships as attributes are a concept used in data models called Functional Data Model.

In object databases, relationships are represented by object references.

In relational databases, foreign keys represent relationships as attributes.

Note: A one-to-one relationship between two entity sets does not imply that for an occurrence of an entity from one set at any time, there must be an occurrence of an entity in the other set.

Chapter 5

Constraints on Relationships

5.1 Cardinality Ratios

The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in.

5.1.1 Types of Cardinality Ratios

- One-to-one (1:1)
- One-to-many (1:N) or Many-to-one (N:1)
- Many-to-many (M:N)

5.2 Participation Constraints

Constraints limit the set of possible combinations of entities that can participate in the relationship type.

5.2.1 Types of Participation Constraints

(a) **Cardinality Ratios**

(b) **Participation Constraints**

(c) Structural constraints (both (a) and (b) taken together)

5.2.2 Cardinality Ratio

The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in.

Example: An employee works in exactly one department and each department should have a minimum 4 employees.

5.2.3 Participation Constraint

If every project has to be associated with a department, then the existence of an instance of Project entity type depends on the existence of an instance of the Handles relationship type. The entity type Project is said to **totally participate** in the relationship type Handles. The relationship is also said to introduce an **existence dependency** on project.

The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type. This constraint specifies the minimum number of relationship instances that each entity can participate in, and is sometimes called the **minimum cardinality constraint**.

Types

(i) Total Participation

(ii) Partial Participation

5.3 Attributes of Relationship Types

5.3.1 Budget

Budget refers to the allocated budget for a specific department for handling a specific project.

If belongs neither to the department nor to the project in its entirety.

Budget refers to the allocated budget for a specific department for handling a specific project.

Example: 1:N or 1:1 relationship attributes can be moved to a corresponding participating entity type.

5.4 Identifying Relationships

The relationship type that relates a weak entity type to its owner is called the identifying relationship.

PAN

5.4.1 Example

Insurance record is a weak entity type with no key. It has no existence without its association with entity type Employee.

The relationship *insurance details* is an identifying relationship.

5.5 Recursive Relationship

A relationship type where the same entity type participates more than once in a different role is called a recursive relationship.

Chapter 6

Enhanced Entity-Relationship (EER)

6.1 Subclasses and Inheritance

An entity class B is said to be a subclass of another entity class A if it shares an "is-a" relationship with A.

Example: "Car is a vehicle" or "Manager is a Employee".

An entity of class B is said to be a specialization of entities of class A. Conversely, entities of class A are generalizations of class B entities.

An entity cannot exist in the database but merely belongs to a subclass. It has to belong to the super class as well.

6.1.1 Superclass/Subclass Relationships

Subclass entities undergo type inheritance of the super class. That is, each member of the subclass has the same attributes as the super class entities and participates in the same relationship types.

Example: A decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the lossless (nonadditive) join property with respect to the set of dependencies F on R if, for every relation state r of R that satisfies F, the following holds where $*$ is the Natural Join of all the relations in D:

$$*(\pi_{R_1}(r), \pi_{R_2}(r), \ldots, \pi_{R_m}(r)) = r$$

The word *loss* refers to loss of information, not to loss of tuples.

6.2 Specialization and Generalization

The process of creating subclasses out of a given entity type is called **specialization**.

The reverse process of taking two or more entity types and clubbing them under a common super class is called **generalization**.

6.3 Predicate-Defined Subclasses

Members can be defined based on the value of an attribute are called predicate-defined subclasses.

6.4 Disjoint and Overlapping Subclasses

Subclasses that are not disjoint are said to overlap.

This is represented by an "o" in the inheritance circle.

Disjoint or overlap may be either partial or total.

6.5 Union Types or Categories

An account holder can be either an Individual or a Family or an Institution, each of whom have their own attributes and keys.

A union type is also called a category.

6.6 Specialization and Generalization Hierarchies and Lattices

A subclass may have further subclasses specified on it, forming a hierarchy or a lattice of specializations.

Reasons for using the concept of specialization:

(1) Some attributes may apply to some but not all entities of the superclass. A subclass is defined in order to group the entities to which these attributes apply

(2) Some relationship types may be participated in only by entities that are members of the subclass

Chapter 7

Alternative Notations for ER Diagrams

7.1 Structural Constraint

This notation involves associating a pair of integers (min, max) with each participation of an entity type E in a relationship type R, where $0 \leq min \leq max$ and $max \geq 1$.

The numbers mean that for each entity e in E, e must participate in at least min and at most max relationship instances in R at any point of time.

Here each employee must work in exactly one department and each department should have minimum 4 employees.

Here $min = 0$ implies partial participation and $min > 0$ implies total participation.

Chapter 8

Relational Model

8.1 Background

The relational model was introduced by Ted Codd of IBM Research in 1970. It introduced the concept of mathematical relation as the underlying basis for the standard data model for most transactional databases today.

8.2 Relational Model Concepts

A relational database is a collection of relations.

A relation resembles a "table" or a "flat file" of records.

Each row or a relational instance is called a tuple, and each column is called an attribute of the relation.

8.3 Some Definitions

A data value is said to be atomic if it cannot be subdivided into smaller data values.

A domain is a set of atomic values.

A relational schema denoted by $R(A_1, A_2, \ldots, A_n)$ is made up of relation name R and a list of attributes $A_1, A_2, \ldots, A_n$. Each A_i is the name of a role played by some domain D in the schema R. The domain of A_i is denoted by $\mathrm{dom}(A_i)$.

The cardinality of the relation is the number of tuples of the relation.

The degree or arity of the relation is the number of attributes of its relation schema. It is also called as Arity.

A relation r of a relation schema $R(A_1, A_2, \ldots, A_n)$ is a set of n-tuples of the form $[t_1, t_2, \ldots, t_n]$ where each $t_i \in \mathrm{dom}(A_i)$.

8.4 Characteristics of Relations

8.4.1 Ordering of Tuples

Tuples in relation don't have any ordering. Mathematically, they do have order when being stored in files or disks.

8.4.2 Ordering of Attributes

Attributes within a tuple have to maintain order if they are stored as tuples. However, a relation need not necessarily be maintained as a set of ordered tuples as long as a mapping between attribute and value is maintained in each relation instance.

8.4.3 Values of Tuples

Each value that makes up a tuple is atomic and cannot be further subdivided. This is also called the First Normal Form assumption.

8.5 Key

A "key" K of a relation is a subset of the superkey $R \subseteq K$ such that removal of any attribute in K removes the superkey property for K. They are also called "minimal superkeys".

A relation schema may have more than one key. Each key is called a candidate key.

Usually one of the candidate keys is chosen to uniquely identify tuples in a relation, such a key is called primary key for a relation.

Chapter 9

Relational Constraints

9.1 Domain Constraints

Domain constraints specify that value of each attribute A must be atomic and a member of $\mathrm{dom}(A)$.

9.2 Key Constraints

Given a relational schema $R = (A_1), i = 1, \ldots, n$, a subset of attributes K in called a superkey if the values of attributes in K is distinct for every relation instance.

This is, for any two tuples t_i, t_j:

$$(t_i \neq t_j) \rightarrow (t_i[K] \neq t_j[K])$$

9.3 Entity Integrity Constraint

The primary key of a tuple may never be null.

9.4 Referential Integrity Constraint

Informally: A tuple that refers to another tuple from another relation should refer to an existing tuple.

Given a relation R_1, a set of attributes FK is said to be a foreign key of R_1 referencing another relation R_2 if the following rules hold:

(a) The attribute in FK have the same domain as the primary key of R_2

(b) For every tuple in R_1, the attribute in its foreign key FK either reference to a tuple in R_2 or is null

Example:

Emp_id	Name	Works-in	Reports-to

Department:

Dept-id	Location	Managed by	Num members

Foreign keys are diagrammatically depicted as arrows.

A foreign key can be from a relation to itself.

9.5 Semantic Integrity Constraints

Constraints on the useless of attributes. Example: the minimum age of an employee is 18 and maximum is 65.

Chapter 10

Relational Algebra and Relational Calculus

Relational algebra is a collection of operations to manipulate relations. The result of each of these relations is also a relation.

Relational algebra is a procedural language. It specifies the operations to be performed on existing relations to derive result relations.

The select operation is used to select (retrieve) a subset of the tuples from a relation that matches the selection condition.

10.1 The SELECT Operation

$$\sigma_{\text{Salary}>3000}(\text{Employee})$$

Selects all tuples of relation Employee which matches the criteria Employee.Salary > 3000.

10.2 Properties of SELECT Operation

The SELECT operation is unary. It operates on only one relation.

Selection conditions are applied to each tuple individually in the relation. Hence the condition cannot span more than one tuple.

The degree of the relation resulting from σ_{CR} is same as that of R.

$|\sigma_{CR}| \leq |R|$

The SELECT operation is commutative:

$$\sigma_{\sigma_1(\sigma_2(R))} \Leftrightarrow \sigma_{\sigma_2(\sigma_1(R))}$$

Hence select operation can be cascaded in any manner.

10.3 The Project Operation

The project operation selects specific columns from the specified relation.

$$\pi_{\text{Name, Salary}}(\sigma_{\text{Salary}>3000}(\text{Employee}))$$

Returns Name and Salary field of all records of Employee where salary is greater than 3000.

10.4 Properties of Project Operation

The number of tuples returned by Project is less than or equal to number of tuples in the specified relation.

When attribute list of PROJECT includes the super key, then the number of tuples returned is equal to the number of tuples in the specified relation.

PROJECT is not commutative:

$$\pi_{L_2}(\pi_{L_1}(R)) \neq \pi_{L_1}(\pi_{L_2}(R))$$

if L_2 is a substring of L_1. Else the operation is an incorrect operation.

10.5 Composition and Assignment

Since the input and output of the relational operators is a single relation, they can be composed in any fashion, with the output of one operation being the input of the other.

$$\pi_{\text{Name, Salary}}(\sigma_{\text{Salary}>3000}(\text{Employee}))$$

Returns Name and Salary field of all records of Employee where salary is greater than 3000.

Results of a query can also be assigned to a name to form a relation by that name.

Salary Statement $\leftarrow \pi_{\text{Name, Salary}}(\sigma_{\text{Salary}>3000}(\text{Employee}))$

Creates a new relation Salary Statement out of the specified query.

10.6 Cartesian Join ($\times$)

The join operator allows the combining of two relations to form a single new relation.

Join is basically the Cartesian product of the relations followed by a selection operation.

Consider the Student and Lab relation:

Student: $\bowtie_{\text{Student.Lab = Lab.Name}}$ **Lab**

Returns the same result as:

$$\sigma_{\text{Student}\times\text{Lab}}(\sigma_{\text{Student.Lab = Lab.Name}})$$

10.7 Properties of Cartesian Join

The Cartesian join where the only comparison operator used is the equality (=) sign are called equijoins.

A natural join denoted by $*$ in an equijoin where the attribute names at both ends of the equality sign are the same. In this case, the attribute is not repeated in the final result.

Consider the Student and the Lab example. Let the Student relation schema is modified as Student(RollNo, Name, LabName, Faculty, Dept).

The natural join Student $*$ Lab has a schema:

(RollNo, Name, LabName, Faculty, Dept)

10.8 Theta Join ($\bowtie_\theta$)

Theta join is used to combine related tuples from two or more relations (specified by the condition θ) to form a single tuple.

10.9 Properties of Theta Join

Theta join where the only comparison operator used is the equality (=) sign are called **equijoins**.

A natural join denoted by $*$ is an equijoin where the attribute names at both ends are of the equality sign are the same. $*$ In this case, the attribute is not repeated in the final result.

10.10 Properties of Cartesian Join

The cartesian join represents a Canonical Join of five or more relations. If relation R having $|R|$ tuples and relation S having $|S|$ tuples are joined using X, then $(R \times S) = |R|.|S|$

The union operator $\cup$ returns the set of all tuples that are present in either R or S or both. Duplicate tuples will be eliminated.

The intersection operator $\cap$ returns the set of all tuples that are present in both R and S.

The Set Difference Operator $R - S$ returns the set of all tuples in R but not in S.

10.11 Compatibility

Two relations $R(A_1, A_2, \ldots, A_k)$ and $S(B_1, B_2, \ldots, B_k)$ are compatible if they have the same number of attributes and for all $A_i, B_i, 1 \leq i \leq k$, $\text{dom}(A_i) = \text{dom}(B_i)$.

Chapter 11

Other Relational Operators

11.1 Division Operator ($\div$)

The division operator is used to denote the conditions where a given relation R is to be split based on whether its association with every tuple in an another relation S.

Let the set of attributes of R be Z and the set of attributes of S be X. Let $Y = Z - X$, that is the set of all attributes of R not in S.

The result of the operation:

$$T(Y) = R(Z) \div S(X)$$

As follows:

R	S
a_1	b_1
a_2	b_1
a_3	b_2

$$T = R - S$$

11.2 Outer Join

A normal join operation by requiring referential integrity. Every attribute in R that refers to S should refer to an existing tuple in S. If the referencing attribute of S is null, the tuple is not returned.

Outer join is used when all tuples are required even when referential integrity is not met. Left outer join includes tuples even when the referencing attribute is null, and Right outer join includes tuples even when the referenced attribute is null or referenced tuple does not exist.

An outer join which is both left and right is called a Full Outer Join.

11.3 Outer Union

Outer union computes the union of partially compatible relations. Two relations $R(X)$ and $S(Z)$ are said to be partially compatible if there exist $W \subseteq X$ and $Y \subseteq Z$ such that for every $A_i \in W$ and $B_i \in Y$, $\mathrm{dom}(A_i) = \mathrm{dom}(B_i)$.

Chapter 12

Relational Calculus

Relational calculus is a query system wherein queries are expressed as variables and formulas on these variables.

A calculus expression specifies what to be retrieved rather than how to retrieve it. Therefore, the relational calculus is considered to be a nonprocedural language.

It has been shown that any retrieval that can be specified in the basic relational algebra can also be specified in the relational calculus and vice versa. In other words, the expressive power of the two languages is identical.

Relational calculus can be categorized in two parts:

(1) Tuple Calculus

(2) Domain Calculus

12.1 Tuple Calculus

Queries in tuple calculus are expressed by a tuple calculus expression. A tuple calculus expression is of the form:

$$\{t|\mathrm{COND}(t)\}$$

where t is a tuple variable and COND(t) is a conditional expression involving t. The result of such a query is the set of all tuples t that satisfy COND(t).

12.2 Informally

Informally, we need to specify the following information in a tuple calculus expression:

(1) For each tuple variable t, the range relation R of t. This value is specified by a condition of the form $R(t)$.

(2) A condition to select a particular combination of tuples

(3) A set of attributes to be retrieved, the requested attributes.

Example: t.attr-name

12.3 Expressions and Formulas in Tuple Relational Calculus

A general expression of the tuple relational calculus is of the form:

$$\{t_1.A_1, t_2.A_2, \ldots, t_n.A_n | \text{COND}(t_1, t_2, \ldots, t_n, t_{n+1}, \ldots, t_m)\}$$

where $t_1, t_2, \ldots, t_n, t_{n+1}, \ldots, t_m$ are tuple variables, each A_i is an attribute of the relation on which t_i ranges, and COND is a condition or formula of the tuple relational calculus. It is also called a well formed formula (wff).

12.4 Formulas (wff) are Built from Atoms

(1) Every atom is a formula

(2) If f and g are formulas then: If $f \vee g$, $f \wedge g$, $f \rightarrow g$ are also formulas

(3) If $f(x)$ is a formula where x is in free, then $\exists x(f(x))$ and $\forall x(f(x))$ are also formulas. However x is now bound.

12.5 The Logical Implication Expression

The logical implication expression:

$$f \rightarrow g$$

means if f then g is equivalent to $\neg f \vee g$.

If the formula $\exists x(f(x))$ is TRUE if the formula $f(x)$ evaluates to true for some (at least one) tuple assigned to free occurrences of x in $f(x)$, otherwise it is false.

If the formula $\forall x(f(x))$ is TRUE if the formula $f(x)$ evaluates to true for every tuple assigned to free occurrences of x in $f(x)$, otherwise it is false.

12.6 Transforming the Universal and Existential Quantifiers

(1) $(\forall x)(P(x)) \equiv \neg(\exists x)(\neg P(x))$

(2) $(\forall x)(P(x)) \equiv (\exists x)(\neg P(x))$

(3) $(\forall x)(P(x)) \equiv \neg(\exists x)(\neg(P(x)))$

(4) $(\exists x)(P(x)) \equiv \neg(\forall x)(\neg P(x))$

(5) $(\forall x)(P(x) \wedge Q(x)) \equiv (\exists x)(\neg P(x) \vee \neg(Q(x)))$

(6) $(\forall x)(P(x) \vee Q(x)) \equiv (\exists x)(\neg P(x) \wedge \neg(Q(x)))$

(7) $(\exists x)(P(x) \wedge Q(x)) \equiv \neg(\forall x)(\neg(P(x)) \vee \neg(Q(x)))$

(8) $(\exists x)(P(x) \vee Q(x)) \equiv \neg(\forall x)(\neg P(x) \wedge \neg(Q(x)))$

(9) $(\forall x)(P(x)) \rightarrow (\exists x)(P(x))$

(10) $\neg(\exists x)(P(x)) \rightarrow \neg(\forall x)(P(x))$

12.7 Safe Expressions

A safe expression in relational calculus is one that is guaranteed to yield a finite number of tuples as its result, otherwise the expression is called unsafe.

An expression is said to be safe if all values in its result are from the domain of the expression E: $\{t|\text{NOT}(\text{Employee}(t))\}$ is unsafe.

12.8 Domain Calculus

Domain calculus differs from tuple calculus in the type of variables used in formulas. Rather than having variables range over tuples, the variables range over single values from domains of attributes.

To form a query result, we must have n of these domain variables — one for each attribute. An expression of the domain relational calculus is of the form:

$$\{x_1, x_2, \ldots, x_n|\text{COND}(x_1, x_2, \ldots, x_n, x_{n+1}, \ldots, x_m)\}$$

where $x_1, x_2, \ldots, x_n, x_{n+1}, \ldots, x_m$ are domain variables that range over domains and COND is a condition or formula of the domain relational calculus.

Note:

SQL is based on tuple relational calculus or while QBE is no related to domain relational calculus. Both of these languages were developed by IBM Research.

SQL - Structured Query Language

QBE - Query by example.

Chapter 13

Relational Completeness

Relational completeness means that a query language can express all the queries that can be expressed in the relational algebra. It does not mean that the language can express any desired query.

Chapter 14

Relational Database Design

14.1 Informal Design Guidelines for Relation Schemas

14.1.1 Semantics of the Relation Attributes

Design a relation schema so that it is easy to explain its meaning. Do not combine attributes from multiple entity types and relationship types into a single relation.

14.1.2 Reducing the Redundant Values in Tuples

Design the base relation schemas so that no insertion, deletion, or modification anomalies are present in the relations. If any anomalies are present, note them clearly and make sure that the programs that update the database will operate correctly.

14.1.3 Reducing the Null Values in Tuples

As far as possible, avoid placing attributes in a base relation whose values may frequently be null. If nulls are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

14.1.4 Disallowing the Possibility of Generating Spurious Tuples

Design relation schemas so that they can be joined together with equality conditions on attributes that are either primary key or foreign key in a way that guarantees that no spurious tuples are generated. Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples.

Chapter 15

Functional Dependencies

15.1 Framework for Automatic Design and Optimization of Relational Schemas

- Generalization over the notion of keys
- Crucial in obtaining correct normalized schemas

15.2 Definition of Functional Dependency

In any relation R, if there exists a set of attributes $A_1, A_2, \ldots, A_n$ and on attribute B such that if any two tuples have the same value for $A_1, A_2, \ldots, A_n$, then they also have the same value for B.

A functional dependency (FD) of the above form is written as:

$$A_1, A_2, \ldots, A_n \rightarrow B$$

Functional dependencies define properties of the schema and not of any particular legal relation state (extension) of R. Hence an FD cannot be inferred automatically from a given relation extension r, but must be defined explicitly by someone who knows the semantics of the attributes of R.

15.3 Trivial and Non-trivial Functional Dependency

Note that in the Movies relation:

title, *year* $\rightarrow$ *title*

An FD where the right-hand side is contained within the left-hand side is called a trivial FD.

Example: $(X \supseteq Y)$

If there is at least one element on R.H.S that is not contained in L.H.S, it is called a non-trivial FD.

15.4 Inference Rules for Functional Dependencies

FDs which are specified are called stated FDs, and FDs which are derived are called inferred FDs.

We use the notation $F \models X \rightarrow Y$ to denote that the functional dependency $X \rightarrow Y$ is inferred from the set of functional dependencies F.

15.4.1 Inference Rule 1: Reflexive Rule

If $X \supseteq Y$, then $X \rightarrow Y$

15.4.2 Inference Rule 2: Augmentation Rule

$\{X \rightarrow Y\} \vdash XZ \rightarrow YZ$

15.4.3 Inference Rule 3: Transitive Rule

$\{X \rightarrow Y, Y \rightarrow Z\} \vdash X \rightarrow Z$

Inference Rules (IR1), (IR2), and (IR3) are sound and complete. These are called Armstrong's Axioms (AA) or (free from defect) Soundness.

Every new FD $X \rightarrow Y$ derived by FD using Armstrong's Axioms from a given set of FDs F using Armstrong's Axioms is such that: $F \models \{X \rightarrow Y\}$

Completeness:

Any FD $X \rightarrow Y$ logically implied by F (i.e., $F \vdash \{X \rightarrow Y\}$) can be derived from F using Armstrong's Axioms.

15.4.4 Inference Rule 4: Decomposition Rule

$\{X \rightarrow YZ\} \vdash X \rightarrow Y, X \rightarrow Z$

15.4.5 Inference Rule 5: Union Rule (Additive Rule)

$\{X \rightarrow Y, X \rightarrow Z\} \vdash X \rightarrow YZ$

15.4.6 Inference Rule 6: Pseudotransitive Rule

$\{X \rightarrow Y, WY \rightarrow Z\} \vdash WX \rightarrow Z$

Proof:

$$\begin{array}{ll} X \rightarrow Y & \text{(given)} \\ WY \rightarrow Z & \text{(given)} \\ WX \rightarrow WY & \text{(using IR2 on 1)} \\ WX \rightarrow Z & \text{(using IR6 on 3 and 2)} \end{array}$$

Inference Rules IR1 through IR3 are known as Armstrong's Inference Rules, or Armstrong's Axioms.

15.5 Specified Functional Dependency

Certain FDs can be specified without referring to a specific relation, but as a property of these attributes.

15.6 Closure of FDs

The set of all dependencies that include F as well as all dependencies that can be inferred from F is called the closure of F. It is denoted by F^+.

15.7 Computing Closures

The size of F^+ can sometimes be exponential in the size of F.

For instance:

$$F = \{A \to B, A \to B_2, \ldots, A \to B_n\}$$
$$F^+ = \{A \to X\}, \text{ where } X \in \{B_1, B_2, \ldots, B_n\}$$

Thus $|F^+| = 2^n$

Since the method of computing F^+ is computationally expensive, however there is an alternative method to check $F \vdash X \to Y$ without generating F^+.

Checking if $X \to Y \in F^+$ can be done by checking if $Y \subseteq X^+$, where X^+ is $*$ the closure of X under F.

15.8 Definition of X_F^+

The closure of X w.r.t. F, written as X_F^+, is the set of all attributes that are eventually defined by $*$ X.

Let:

$$A \to B, B \to C, D \to E$$

Then:

$$A^+ = A \cup B \cup C \cup D \cup E$$

15.9 Algorithm to Compute X^+

Algorithm to compute X^+ w.r.t F for each FD $X \to Y$ in F:

(1) Initially $X^+ = X$

(2) For every $X' \subseteq X$, if there exists an FD of the form $X' \to Y$ and $Y \not\subseteq X$, then $X^+ = X' \cup Y$

(3) Repeat step 2 until no more attributes can be added to closure X^+

15.10 Covers and Equivalence of FDs

A set of functional dependencies F is said to cover another set of functional dependencies E if every FD in E is also in F^+, that is, if every dependency in E can be inferred from F; alternatively, we can say that E is covered by F^+.

Equivalence:

Two sets of functional dependencies E and F are equivalent if $E^+ = F^+$. Hence, equivalence means that every FD in E can be inferred from F, and every FD in F can be inferred from E; that is, E is equivalent to F if both the conditions E covers F and F covers E hold.

15.11 Determining whether F covers E

A functional dependency is a property of the relation schema (intension) R, not of a particular legal relation state (extension) of R. Hence an FD cannot be automatically inferred from a given relation extension r, but must be defined explicitly by someone who knows the semantics of the attributes of R.

15.12 Algorithm to Find a Non-Redundant Cover

(1) Initially $E = F$

(2) Replace each functional dependency $X \to \{A_1, A_2, \ldots, A_n\}$ in F by the n functional dependencies $X \to A_1, X \to A_2, \ldots, X \to A_n$

(3) For each functional dependency $X \to A$ in F, for each attribute B that is an element of X, if $\{F - \{X \to A\}\} \cup \{(X - \{B\}) \to A\}$ is equivalent to F, then replace $X \to A$ with $(X - \{B\}) \to A$ in F

(4) For each remaining FDs $X \to A$ in F, if $\{F - \{X \to A\}\}$ is equivalent to F, then remove $X \to A$ from F

(5) $E = F$

15.13 Canonical Cover or Minimal Cover

A set of FDs F is called a canonical cover or minimal cover if it satisfies the following conditions:

(1) Every dependency in F has a single attribute for its right-hand side

(2) We cannot replace any dependency $X \to A$ in F with a dependency $Y \to A$ where $Y \subset X$ and still have a set of dependencies that is equivalent to F

(3) We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F

If F is a set of FDs and if F is a nonredundant cover of F, then it is not true that E has the minimum number of FDs.

15.14 Full Functional Dependency

Given a relational schema R and an FD $X \to Y$, if Y is fully functionally dependent on X if there is no Z, where $Z \subseteq X$ such that both $X \to Z$ and $Z \to Y$ hold.

Chapter 16

Normalization of Relations

Normalization of data can be looked upon as a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of:

(1) Minimizing redundancy

(2) Minimizing the insertion, deletion, and update anomalies

The normal form of a relation refers to the highest normal form condition that it meets.

16.1 First Normal Form (1NF)

A relation schema R is said to be in First Normal Form (1NF) if the values in the domain of each attribute of the relation are atomic.

In other words, only one value is associated with each attribute and the value is not a set of values or a list of values.

16.2 Second Normal Form (2NF)

Second Normal Form (2NF) is based on the concept of full functional dependency.

A functional dependency $X \to Y$ in a relation schema R is a transitive dependency if there is a set of attributes Z that is neither a candidate key nor a subset of any key of R, and both $X \to Z$ and $Z \to Y$ hold.

A relation schema R is in 2NF if every non-prime attribute A in R is fully functionally dependent on the primary key of R.

16.3 Third Normal Form (3NF)

3NF is based on the concept of transitive dependency.

A functional dependency $X \to Y$ in a relation schema R is a transitive dependency if there is a set of attributes Z in R is neither a candidate key nor a subset of any key of R, and both $X \to Z$ and $Z \to Y$ hold.

16.3.1 General Definition of 2NF

A relation schema R is in 2NF if every non-prime attribute A in R is not partially dependent on any key of R.

16.3.2 General Definition of 3NF

A relation schema R is in third normal form (3NF) if, whenever a non-trivial functional dependency $X \rightarrow A$ holds in R, either:

(a) X is a superkey of R, or

(b) A is a prime attribute of R

16.4 Boyce-Codd Normal Form (BCNF)

According to Codd's original definition, a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

Emp-Dept:

ENAME	ENO	DOB	ADDRESS	DNUMBER	DNAME	DMGRENO

3NF Normalization:

ENO	PNUMBER	HOURS

FD2:

PNUMBER	PNAME	PLOCATION

FD3:

ENO	ENAME

Third Normal Form: (Considers only primary keys)

3NF is based on the concept of transitive dependency.

A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there is a set of attributes Z that is neither a candidate key nor a subset of any key of R, and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

16.4.1 Note

In general definitions of 2NF and 3NF, partial and full functional dependencies and transitive dependencies are considered with respect to all candidate keys of a relation.

16.4.2 General Definition of 2NF (Candidate Keys)

A relation schema R is in 2NF if every nonprime attribute A in R is fully functionally dependent on every key of R.

16.4.3 General Definition of 3NF (Candidate Keys)

A relation schema R is in third normal form (3NF) if, whenever a nontrivial functional dependency $X \rightarrow A$ holds in R, either (a) X is a superkey of R, or (b) A is a prime attribute of R.

16.4.4 Note

In the definition of normal forms, an attribute that is part of any candidate key will be considered as prime attribute.

16.4.5 2-attribute Relations

Any 2-attribute relation of the form $R(A, B)$ is always in BCNF.

Proof: Consider the following cases:

(1) There are no FDs between A and B, in which case only trivial FDs exist and R is in BCNF

(2) $A \rightarrow B$ but there is no FD of the form $B \rightarrow A$. In this case, A is a key and R is in BCNF

(3) $B \rightarrow A$ and $B \rightarrow A$. Both A and B are keys, thus does not violate the BCNF condition

(4) $A \rightarrow B$ and $B \rightarrow A$. Both A and B are keys, thus does not violate the BCNF condition

Chapter 17

Properties of Relational Decompositions

17.1 Attribute Preservation Property of a Decomposition

Let R be a relation schema and let $D = \{R_1, R_2, \ldots, R_m\}$ be the decomposition of R into $R_1, R_2, \ldots, R_m$. Then:

We must make sure that each attribute in R will appear in at least one relation schema R_i in the decomposition so that no attributes are lost.

Formally, we have:

$$\bigcup_{i=1}^{m} R_i = R$$

This is called the attribute preservation property of decomposition.

17.2 Dependency Preservation Property of a Decomposition

If each functional dependency $X \to Y$ specified in F either appears directly in one of the relation schemas R_i in the decomposition D or could be inferred from the dependencies that appear in some R_i.

A decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R is dependency preserving with respect to F if the union of the projections of F on each R_i in D is equivalent to F; that is:

$$((\pi_{R_1}(F)) \cup (\pi_{R_2}(F)) \cup \ldots \cup (\pi_{R_m}(F)))^+ = F^+$$

17.3 Lossless (Nonadditive) Join Property of a Decomposition

If F is a set of FDs on R, all legal relation states r of R that satisfy F must also satisfy the lossless join property of decomposition. If a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the lossless (nonadditive) join property with respect to the set of dependencies F

on R if, for every relation state r of R that satisfies F, the following holds where $*$ is the Natural Join of all the relations in D:

$$*(\pi_{R_1}(r), \pi_{R_2}(r), \ldots, \pi_{R_m}(r)) = r$$

The word *loss* refers to loss of information, not to loss of tuples.

17.4 Algorithm for Testing for Lossless (Nonadditive) Join Property

Input: A universal relation R, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R, and a set F of functional dependencies.

(1) Create an initial matrix S with one row i for each relation R_i in D and one column j for each attribute A_j in R

(2) Set $S(i, j) = a_j$ for all matrix entries

(3) For each row i representing relation schema R_i, if relation R_i includes attribute A_j, then set $S(i, j) = a_j$

(4) Repeat the loop until a complete loop execution results in no change to S: for each FD $X \to Y$ in F, for each column j representing attribute A_j, if make the symbols in each column that correspond to an attribute in Y be the same in all three rows or follows

If relation R_i includes attribute A_j, then set $S(i, j) = a_j$.

If a row is not made up entirely of "a" symbols, then the decomposition has the lossless join property, otherwise it does not.

17.5 Successive Lossless (Nonadditive) Join Decompositions

If a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the lossless (nonadditive) join property with respect to a set of F-s F on R, and if a decomposition $D_i = \{Q_1, Q_2, \ldots, Q_k\}$ of R_i has the nonadditive join property w.r.t. the projection of F on R_i, then the decomposition $D_2 = \{R_1, R_2, \ldots, R_{i-1}, Q_1, Q_2, \ldots, Q_k, R_{i+1}, \ldots, R_m\}$ of R has the nonadditive join property w.r.t. F.

17.6 Testing Binary Decomposition for the Nonadditive Join Property

A decomposition $D = \{R_1, R_2\}$ of R has the lossless join property w.r.t. a set of FDs F on R if and only if either:

The FD $(R_1 \cap R_2) \to (R_1 - R_2)$ is in F^+, or

The FD $(R_1 \cap R_2) \to (R_2 - R_1)$ is in F^+

Chapter 18

Algorithms for Creating a Relational Decomposition

18.1 Dependency-Preserving Decomposition into 3NF Schemas

Input: A universal relation R and a set of FDs F on the attributes of R.

(1) Find a minimal cover G for F

(2) For each left-hand side X of an FD that appears in G, create a relation schema in D with attributes $\{X \cup \{A_1\} \cup \{A_2\} \cup \ldots \cup \{A_k\}\}$, where $X \to A_1, X \to A_2, \ldots, X \to A_k$ are the only dependencies in G with X as left-hand side (X is the key of this relation)

(3) If none of the relation schemas in D contains a key of R, then create one more relation schema in D that contains attributes that form a key of R

18.2 Algorithm to Find a Key K of R

Input: A relation R and a set of FDs F.

(1) Set $K = R$

(2) For each attribute A in K, compute $(K - A)^+$ w.r.t. F. If $(K - A)^+$ contains all attributes in R, then set $K = K - \{A\}$

18.3 Lossless (Nonadditive) Join Decomposition into BCNF Schemas

Input: A universal relation R and a set of FDs F.

(1) Set $D = \{R\}$

(2) While there is a relation schema Q in D that is not in BCNF, do:

  (i) Choose a relation schema Q in D that is not in BCNF
  (ii) Find an FD $X \to Y$ in Q that violates BCNF
  (iii) Replace Q in D by two relation schemas $(Q - Y)$ and $(X \cup Y)$

18.4 We Use 3NF if We Require Efficiency

We use 2NF if we have more data retrieval operations (SELECT), then insert, update, or delete operations.

18.5 Normal Forms are are Insufficient on Their Own as Criteria for a Good Relational Database Schema Design

The relation must collectively satisfy these two additional properties:

(i) Dependency Preservation Property

(ii) Lossless or nonadditive join property

Chapter 19

Nested Loop Join and Block Nested Join

19.1 Nested Loop Join

For each tuple r in R do
For each tuple s in S do
If r and s satisfy the join condition, then output the tuple $\langle r, s\rangle$
This algorithm involves $n_r \times b_s + b_r$ block transfers, where:
n_r = No. of records in R
b_s = No. of blocks used by S

19.2 Block Nested Loop Join

For each block b in R do
For each tuple s in S do
For each tuple r in block b do
If r and s satisfy the join condition, then output the tuple $\langle r, s\rangle$
This algorithm involves $b_r \times b_s + b_r$ block transfers.

Chapter 20

Transaction Processing and Concurrency Control

20.1 Properties of Transaction (ACID)

(1) **Atomicity:** Transaction Management Component

(2) **Consistency**

(3) **Isolation:** Concurrency Control Component

(4) **Durability:** Recovery Management Component

20.2 Transaction State

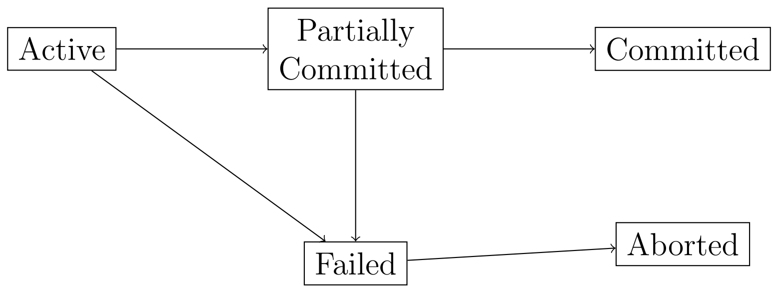
(1) Active: While executing

(2) Partially Committed: After the final statement has been executed

(3) Failed: When normal operation can no longer proceed

(4) Aborted: After the transaction has been rolled back and database has been restored to its state prior to start of the transaction

(5) Committed: After successful completion



Terminated means if the transaction has either committed or aborted.

20.3 Concurrent Execution of Transactions Allows

(1) Improved throughput and resource utilization

(2) Reduced waiting time

20.4 Schedules

Schedules represent the order in which instructions are executed in the system.

20.5 Serial Schedule

It consists of a sequence of instructions from various transactions, where the instructions belonging to one single transaction appear together in that schedule.

20.6 Example

T_1	T_2
Read (A)	
A = A - 50	
Write (A)	
Read (B)	
B = B + 50	
Write (B)	
	Read (A)
	Write (A)
	Read (B)
	Write (B)
	Read (C)
	Write (C)

20.7 Conflict Serializability

Two instruction I_i and I_j conflict if they are operations by different transactions on the same data item and at least one of these instructions is a write operation.

If a schedule S can be transformed into a schedule S' by a series of swaps of nonconflicting instructions, we say that S and S' are conflict equivalent.

A schedule S is conflict serializable if it is conflict equivalent to a serial schedule.

20.8 View Serializability

The schedules S and S' are said to be view equivalent if three conditions are met:

(1) For each data item Q, if transaction T_i reads the initial value of Q in schedule S, then transaction T_i must, in schedule S', also read the initial value of Q

(2) For each data item Q, if transaction T_i executes read (Q) in schedule S, and if that value was produced by a write (Q) operation executed by transaction T_j, then the read (Q) operation of transaction T_i must, in schedule S', also read the value of Q that was produced by the same write (Q) operation of transaction T_j

(3) For each data item Q, the transaction (if any) that performs the final write (Q) operation in schedule S must perform the final write (Q) operation in schedule S'

A schedule S is view serializable if it is view equivalent to a serial schedule.

20.9 Example

T_3	T_4	T_6
Read (Q)		
	Write (Q)	
Write (Q)		
		Write (Q)

Every conflict-serializable schedule is also view-serializable, but there are view-serializable schedules that are not conflict serializable.

20.10 Recoverable Schedules

A recoverable schedule is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the commit operation of T_j.

20.11 Cascadeless Schedules

The phenomenon in which a single transaction failure leads to a series of transaction rollbacks is called cascading rollback.

A cascadeless schedule is one where for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the read operation of T_j.

Every cascadeless schedule is also recoverable.

20.12 Testing for Conflict Serializability

Construct a directed graph called a precedence graph from schedule S. This graph consists of a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges.

The set of vertices consists of all the transactions participating in the schedule.

The set of edges consists of all edges $T_i \rightarrow T_j$ for which one of three conditions hold:

(1) T_i executes write (Q) before T_j executes read (Q)

(2) T_i executes read (Q) before T_j executes write (Q)

(3) T_i executes write (Q) before T_j executes write (Q)

If an edge $T_i \rightarrow T_j$ exists in the precedence graph, then in any serial schedule S' equivalent to S, T_i must appear before T_j.

If the precedence graph for S has a cycle, then schedule S is not conflict serializable.

20.13 Example

T_1	T_2
Read (A)	
Write (A)	
	Read (A)
	Write (A)
	Read (C)
	Write (C)
Read (B)	
Write (B)	

Recovery:
T_1: No changes T_2: Redo C, T_3: Redo ABC
T_4: Undo AB T_5: Discard AB

Chapter 21

Transaction Log

21.1 Techniques to Maintain Log Files

(1) Deferred Update

(2) Immediate Update

21.2 Occurrence of Failure in Deferred Update Scheme

$\langle T_0$ Start$\rangle$	$\langle T_0$ Start$\rangle$	$\langle T_0$ Start$\rangle$
$\langle T_0, A, 950\rangle$	$\langle T_0, A, 950\rangle$	$\langle T_0, A, 950\rangle$
$\langle T_0, B, 2050\rangle$	$\langle T_0, B, 2050\rangle$	$\langle T_0, B, 2050\rangle$
CRASH	$\langle T_0$ Commit$\rangle$	$\langle T_0$ Commit$\rangle$
	$\langle T_1$ Start$\rangle$	$\langle T_1$ Start$\rangle$
Recovery:	$\langle T_1, C, 600\rangle$	$\langle T_1, C, 600\rangle$
No Action	CRASH	$\langle T_1$ Commit$\rangle$
		CRASH
	Recovery:	Recovery:
	Redo CT_0	Redo CT_0
		Redo CT_1

21.3 Occurrence of Failure in Immediate Update Scheme

Checkpoints:

$\langle T_0$ Start$\rangle$
$\langle T_0, A, 950\rangle$
$\langle T_0, B, 2050\rangle$
$\langle T_0$ Commit$\rangle$

$\langle T_1$ ABC
T_1: AB
T_2: AB
T_3: C
T_4: ABC
T_5: AB

Recovery:
T_1: No changes T_2: Redo C, T_3: Redo ABC
T_4: Undo AB T_5: Discard AB